

Chapter 15 -- Creating Subclasses: Building Semantic Taxonomy in Ontology

- [15.1 Chapter Introduction -- Why Subclasses Matter](#)
- [15.2 From Flat Classes to Semantic Taxonomy](#)
- [15.3 Understanding Subclasses in OWL](#)
- [15.4 Interesting Reading -- The Mathematical Meaning of Subclasses](#)
- [15.5 Creating Subclasses in Protégé](#)
- [15.6 Semantic Inheritance](#)
- [15.7 Subclass vs. Instance -- Common Beginner Confusion](#)
- [15.8 Graph Database Perspective -- Hierarchy in Neo4j vs. Ontology](#)
- [15.9 EKA Perspective -- Taxonomy as Semantic Compression](#)

15.1 Chapter Introduction -- Why Subclasses Matter

After completing Chapter (14), you have entered a much deeper stage of ontology engineering.

The previous chapter introduced:

- semantic restrictions,
- logical requirements,
- reasoning boundaries, and
- formal inference behavior.

Ontology was no longer merely about representing knowledge structures.

It became increasingly concerned with:

semantic logic.

In this chapter, we return to a concept that may initially appear much simpler:

subclasses.

Inside Protégé, creating a subclass is mechanically straightforward.

You typically:

1. select a parent class,
2. click *Add subclass*, and
3. assign a class name.

However, the semantic significance of subclass modeling is far greater than its simple user interface.

Subclasses provide the foundation for:

- taxonomy,
- specialization,
- inheritance, and

- classification reasoning.

Without subclass hierarchies, ontology quickly becomes:

- flat structure,
- difficult to navigate, and
- semantically weak.

Subclass modeling therefore represents one of the most fundamental building blocks of ontology engineering.

15.2 From Flat Classes to Semantic Taxonomy

Consider a knowledge model where all concepts exist at the same level:

```
Pizza
NamedPizza
VegetarianPizza
SpicyPizza
MargheritaPizza
```

Although these concepts are present, the model lacks semantic structure.

Nothing explicitly indicates:

- which concepts are more generalized
- which are more specialized
- how they relate conceptually

This is known as:

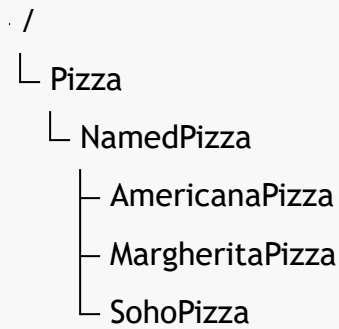
flat classification.

Flat classification may be sufficient for small datasets, but it scales poorly when knowledge becomes complex.

Ontology addresses this problem using:

taxonomy.

A taxonomy organizes concepts hierarchically.



This hierarchy immediately conveys semantic relationships message to the viewers.

Taxonomy therefore provides not only organization, but also:

semantic context.

This context is essential for reasoning.

15.3 Understanding Subclasses in OWL

In ontology engineering, subclass relationships are commonly expressed using:

is-a relationships.

For example: `MargheritaPizza is a NamedPizza`

This means:

`MargheritaPizza is a type of NamedPizza`

Not:

`MargheritaPizza contains NamedPizza`

Nor:

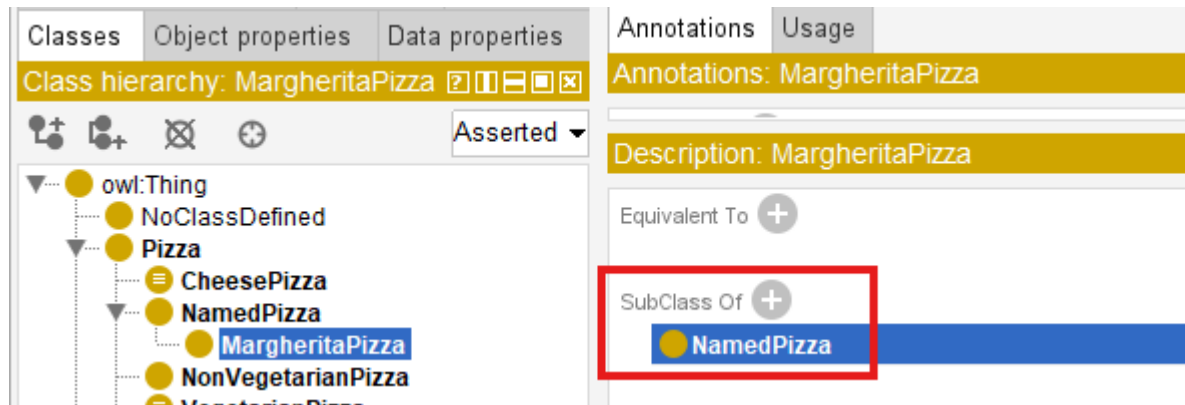
`MargheritaPizza1 references NamedPizza`

Instead, it expresses:

semantic specialization.

In OWL, subclass relationships are represented through `SubClassOf`.

An example: `MargheritaPizza SubClassOf NamedPizza`



Semantically, this states:

every **MargheritaPizza** is also a **NamedPizza**.

This is a strong logical assertion.

Subclass relationships therefore carry formal meaning, not merely visual hierarchy.

15.4 Interesting Reading -- The Mathematical Meaning of Subclasses

To fully understand subclasses, it helps to view them through mathematics.

Ontology class semantics closely align with:

set theory.

A class can be interpreted as:

a set of individuals sharing common characteristics.

Now suppose:

P = set of all pizzas

We can define:

N = set of all named pizzas

This means:

every element of N is also an element of P .

This is the formal meaning of subclass.

More generally:

If :

A = parent class

B = *subclass*

Then :

$B \sqsubseteq A$

Equivalent logical notation is:

$$\forall x (x \in B \rightarrow x \in A)$$

This reads as:

For every x , if x belongs to subclass B , then x must also belong to class A .

This mathematical definition explains why subclass reasoning works.

Suppose:

`myPizza` $\in$ `MargheritaPizza`

And ontology defines:

`MargheritaPizza` $\sqsubseteq$ `NamedPizza`

`NamedPizza` $\sqsubseteq$ `Pizza`

Then by transitivity:

`myPizza` $\in$ `Pizza`

The reasoner performs exactly this type of logical inference automatically.

This is one of the earliest examples showing how ontology enables:

machine reasoning through formal mathematics.

In other words, subclass hierarchy is not merely a tree.

It is formally defined semantic inclusion system.

15.5 Creating Subclasses in Protégé

In the `Pizza` tutorial, the next step is to expand the ontology by creating subclasses under existing classes.

The workflow in Protégé is straightforward.

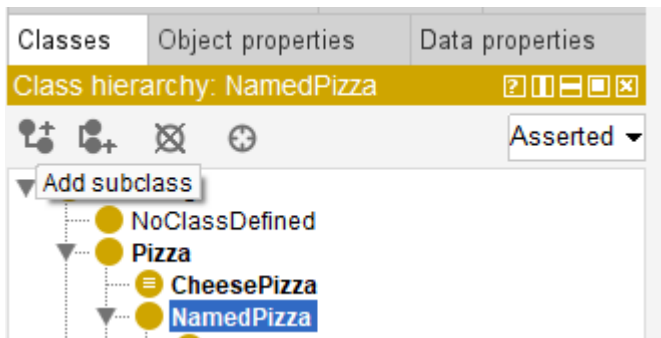
First: select a parent class.

For example: `NamedPizza`

Then choose:

Add subclass

Protégé creates a new child class.



Although visually simple, this action generates a formal OWL axiom:

```
NewClass SubClassOf NamedPizza
```

This expands the ontology's semantic hierarchy.

Each new subclass increases the expressiveness of the ontology.

As more specialized pizza types are introduced, the ontology becomes better suited for:

- classification,
- reasoning, and
- semantic discovery.

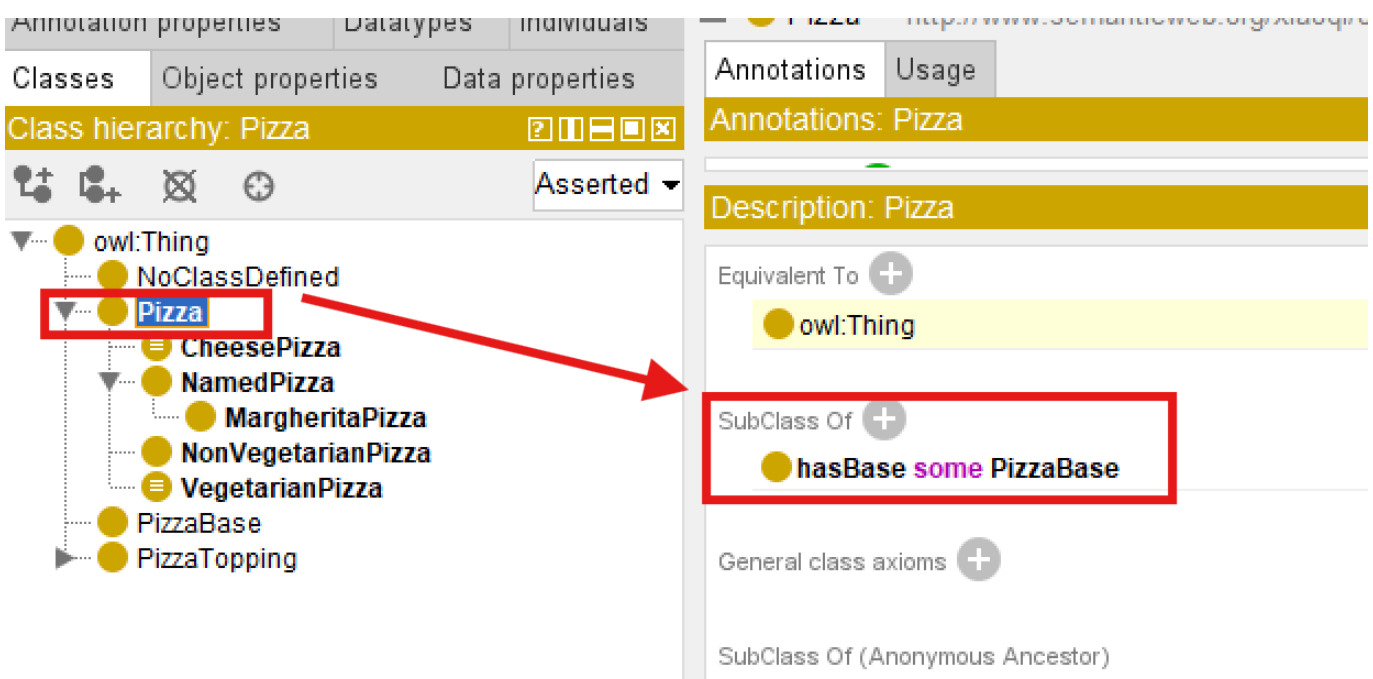
15.6 Semantic Inheritance

One of the most powerful consequences of subclass modeling is:

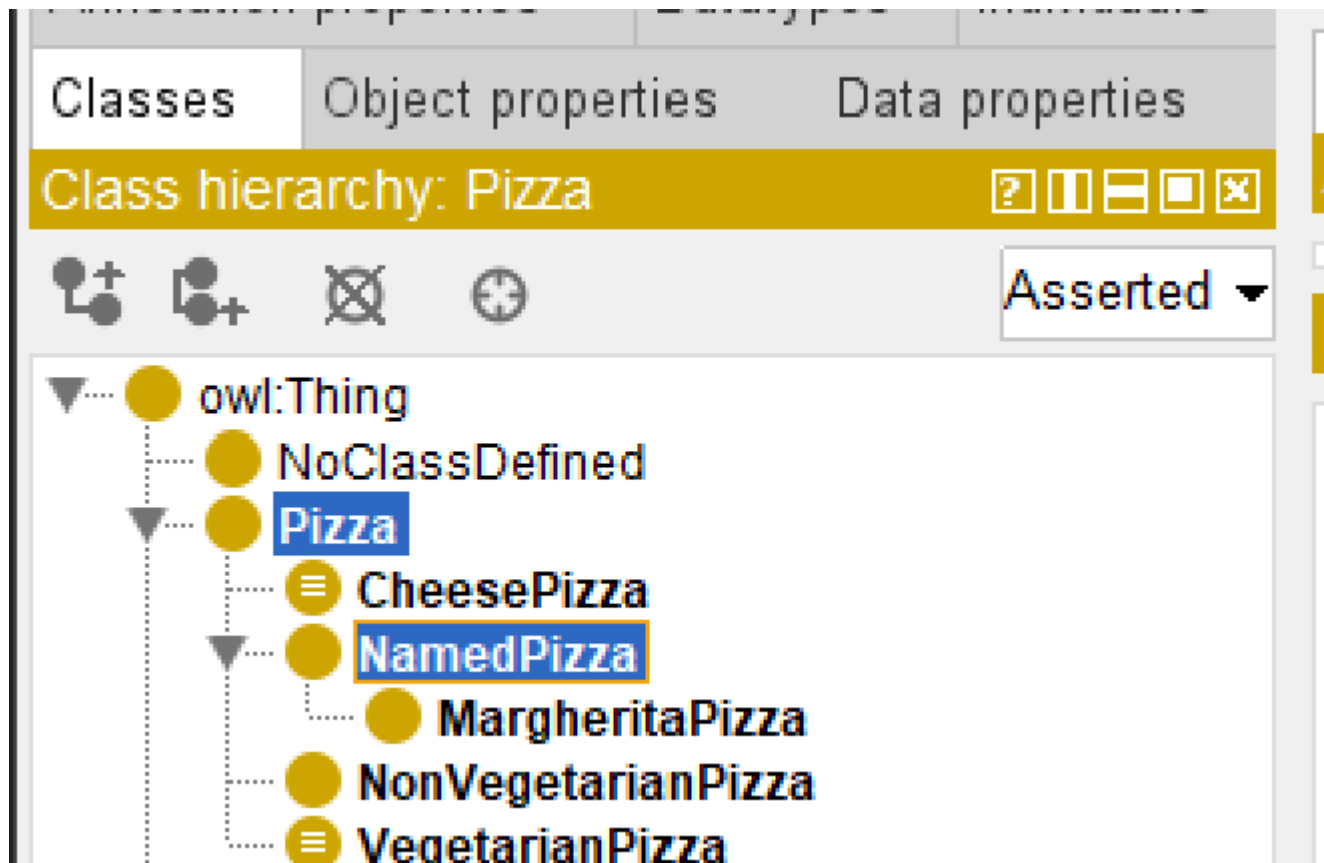
inheritance.

Inheritance means subclasses automatically receive semantic meaning from parent classes.

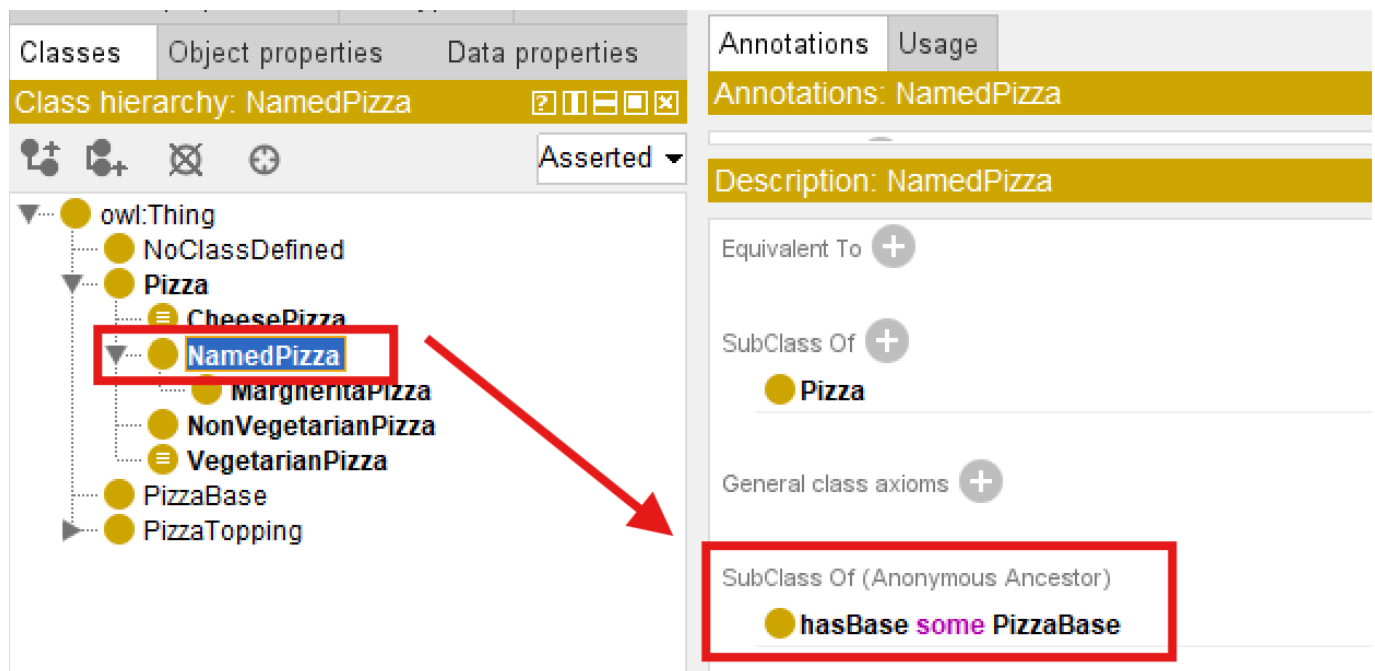
Support ontology defines: `Pizza SubClassOf hasBase some PizzaBase`



Now consider: `NamedPizza SubClassOf Pizza`



Then **NamedPizza** automatically inherits: **hasBase some PizzaBase**



No additional modeling is required.

This greatly reduces redundancy.

Inheritance enables ontologies to scale efficiently while preserving consistency.

This principle becomes increasingly important in enterprise knowledge graphs.

15.7 Subclass vs. Instance -- Common Beginner Confusion

One of the most common beginner mistakes is confusing **subclasses** with **instances**.

Consider: **MargheritaPizza**

Should it be modeled as a class? Or as an individual?

The answer depends on intent.

If it represents:

a recipe category,

then it should be modeled as:

a subclass.

If it represents:

one specific pizza one a table,

then it should be modeled as:

an individual.

This distinction is critical.

Ontology engineers must constantly ask:

Am I modeling a category or a concrete entity?

Correct semantic modeling depends on this decision.

Besides Michael's **Pizza.owl** tutorial, another official document worth to read in detail is the **Ontology 101** (https://protege.stanford.edu/publications/ontology_development/ontology101.pdf), authored by Natalya F. Noy and Deborah L. McGuinness.

I learned quite insightful distinguish between an instance or a class from its Chapter 4.6, and quote following criteria for your reference:

Individual instances are the most specific concepts represented in a knowledge base.

If concepts form a natural hierarchy, then we should represent them as classes (subclasses).

15.8 Graph Database Perspective -- Hierarchy in Neo4j vs. Ontology

Subclass modeling also highlights an important distinction between **graph database** and **ontologies**.

In Neo4j, hierarchy may be stored as graph relationships:

```
CREATE (n:Class {name:"NamedPizza"})
CREATE (p:Class {name:"Pizza"})
CREATE(n)-[:SUBCLASS_OF]->(p)
```

This stores hierarchy structurally.

However, Neo4j itself does NOT automatically understand:

set inclusion semantics

unless EXPLICIT rules are implemented.

OWL reasoners behave differently.

They interpret subclass relationships logically.

This means:

- Graph database stores hierarchy.
- Ontology reasons over hierarchy.

This distinction is essential for enterprise architects building semantic systems.

15.9 EKA Perspective -- Taxonomy as Semantic Compression

From the perspective of Executable Knowledge Architecture (EKA):

$EKA = (K, R, \Theta, \Phi, \Gamma)$,

subclass hierarchies strengthen multiple layers.

K - Knowledge Graph Layer

Subclass hierarchy enriches semantic graph structure and conceptual organization.

R - Reasoning & Rules Layer

Inheritance enables automatic classification and rule propagation.

Γ - Governance Layer

Taxonomy standardizes classification boundaries and semantic consistency.

A useful way to understand subclass hierarchy is as:

semantic compression.

Instead of repeatedly defining semantics for every concept, ontology allows shared semantics to be inherited through hierarchy.

This dramatically improves scalability.

Taxonomy therefore acts as a compression mechanism for semantic knowledge.

Last updated at: 2026-06-28